

Network Bottlenecks in Distributed Training

Find them. Prove them. Fix them.

A practical guide for ML engineers and infrastructure architects facing distributed training performance challenges



Symptoms of a Network Bottleneck

Low GPU Utilization

GPUs show high wall-clock step time despite low compute utilization

Scaling Plateaus

Throughput stops increasing after adding more nodes

Step-Time Variance

Large p95/p99 outliers creating "straggler" effects

Collective Delays

NCCL/Horovod logs show extended All-Reduce or All-Gather phases



Healthy vs. Bottlenecked GPU Utilization

Where Bottlenecks Hide

Intra-node Connections

PCIe vs **NVLink/NVSwitch**
saturation affecting GPU-to-GPU
transfers

Host NIC Interface

GPUDirect RDMA not enabled or
incorrect NUMA configuration
causing extra copies

Rack Fabric

Oversubscribed Top-of-Rack
switches or faulty/inconsistent
cabling

Spine/Core Network

Microbursts, queue drops,
Priority Flow Control (PFC) storms
affecting traffic

Storage/Data Loader

Remote filesystem bottlenecks or
cross-AZ data fetches creating
pipeline stalls

Collective Communications 101

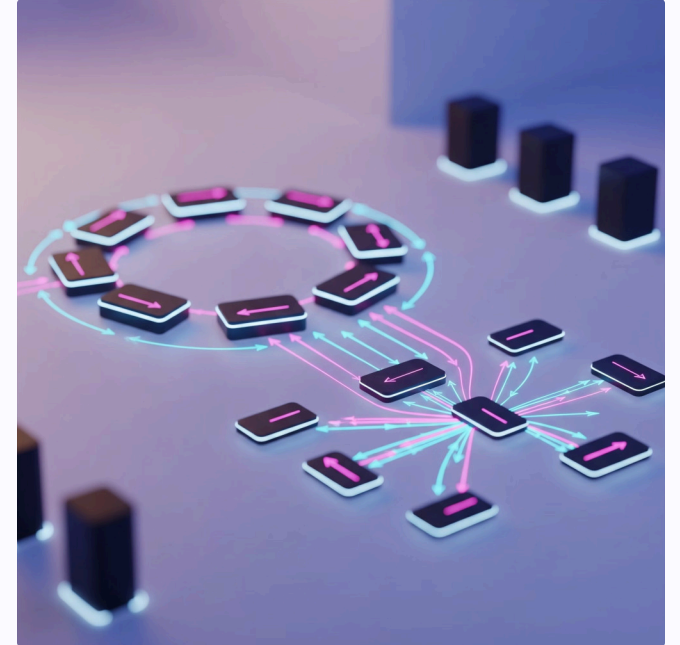
Traffic Patterns

Distributed training relies on **many-to-many** communication patterns:

- **All-Reduce:** Aggregate gradients across all workers
- **All-Gather:** Collect model shards from each worker
- **Broadcast:** Share updated parameters with all workers

Algorithm Tradeoffs

- **Ring:** Bandwidth-efficient, but latency scales with node count
- **Tree/Hierarchical:** Lower latency, performs better at scale



Message Size Impact:

- **Small messages:** Latency-bound
- **Large messages:** Bandwidth-bound

Topology & Placement Matter

Placement Best Practices

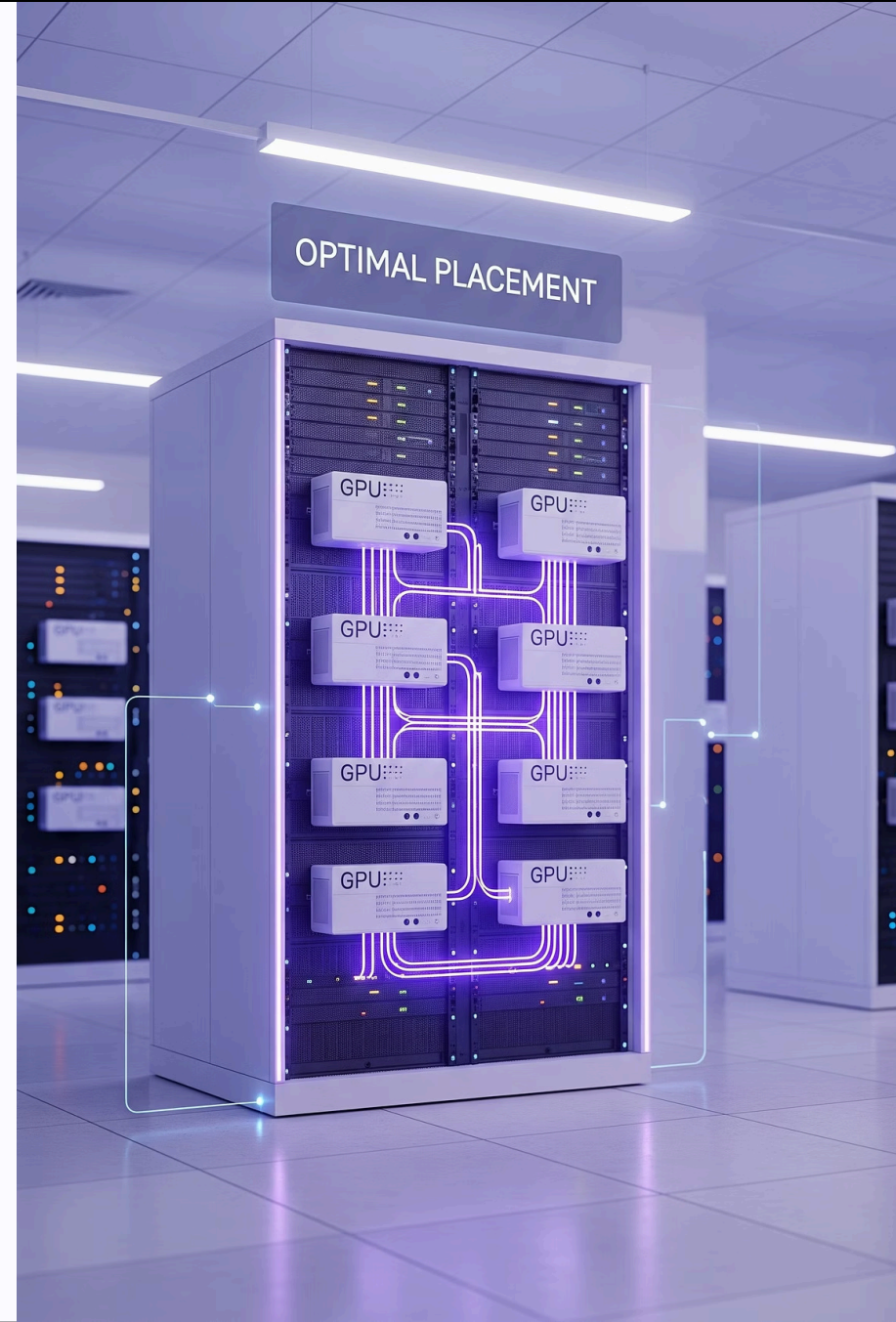
- Keep worker nodes **within the same rack/leaf** when possible
- Map ranks to match physical topology (power-of-two node counts help)
- Use `nvidia-smi topo -m` to verify and align **GPU↔NIC** with NUMA domains

Process Pinning

Follow the 1:1 principle: 1 GPU = 1 process

Set proper **CPU affinity** to ensure process/thread placement aligns with:

- GPU NUMA locality
- NIC PCIe attachment
- Memory controller access





Transport Tuning

InfiniBand / RoCE / Ethernet Optimizations



Enable RDMA

Verify GPUDirect RDMA path is functional end-to-end



Set MTU 9000

Apply jumbo frames consistently across hosts, NICs, and switches



Congestion Control

Configure ECN/DCQCN and use PFC carefully on lossless traffic classes



Watch Buffers

Monitor queue depths and avoid mixing link speeds in a communication ring

NCCL/Horovod Knobs that Matter

Framework Settings

- **PyTorch DDP:** Tune `bucket_cap_mb` (25-100 MB), set `static_graph=True`
- **Horovod:** Adjust `HOROVOD_FUSION_THRESHOLD`, `HOROVOD_CYCLE_TIME`
- Overlap compute/communication (DDP default)
- Use `no_sync()` for gradient accumulation

Environment Variables

```
# Interface selection
NCCL_SOCKET_IFNAME=ib0,eth1
NCCL_IB_HCA=mlx5_0,mlx5_1

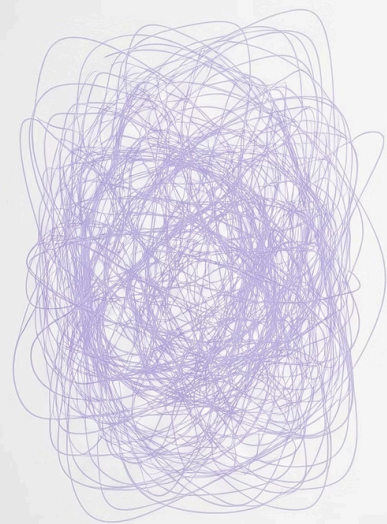
# Protocol settings
NCCL_IB_GID_INDEX=3 # RoCEv2
NCCL_PROTO=LL128    # small msgs
NCCL_NET_GDR_LEVEL=2 # enable GDR
```

 These settings directly impact collective communication performance in distributed training. Start with the defaults, then tune incrementally while measuring impact.

Reduce Bytes on the Wire

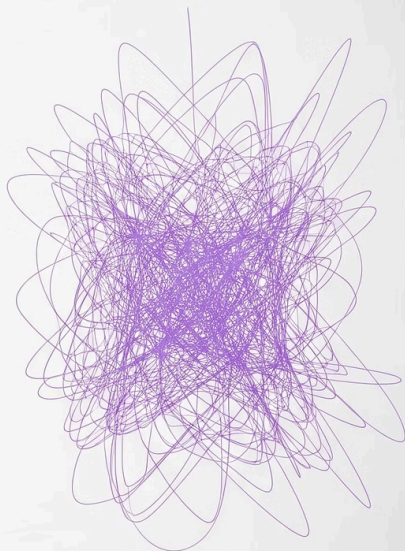
Before Compression

1.2 TB



After Compression

0.3 TB



Precision Reduction

AMP (FP16/BF16) cuts gradient size in half with minimal accuracy impact

Message Optimization

Gradient bucketing/fusion combines small tensors into fewer, larger messages for better network efficiency

Compression Techniques

Apply **8-bit quantization** or other gradient compression methods where accuracy allows

Parameter Sharding

FSDP/ZeRO techniques shard optimizer states to reduce cross-node traffic volume

Reducing communication volume directly improves scaling efficiency in bandwidth-constrained environments

Data Pipeline Isn't Innocent

Optimizing Data Access

Local Caching

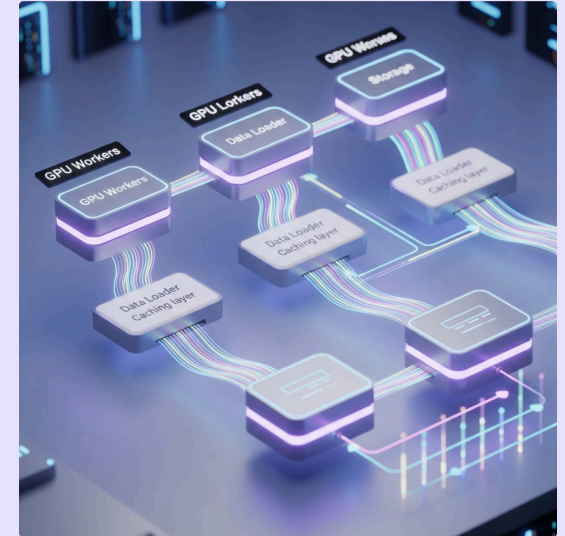
Shard datasets and **prefetch to NVMe** locally to avoid repeated remote reads

Filesystem Selection

Choose **Lustre/BeeGFS** over NFS for scale; warm caches before training

Placement Awareness

Avoid **cross-region/AZ** reads;
schedule jobs near data storage



Hidden Network Impact: High dataloader wait times create communication idle periods that manifest as apparent "network" bottlenecks

Diagnostics: Prove It's the Network

1

NCCL Tests

Run `all_reduce_perf` across varying node counts and message sizes to establish baseline performance

```
$ all_reduce_perf -b 8 -e 128M -f 2 -g 8
```

2

Raw Link Checks

Use `iperf3`, `ib_write_bw`, and `ib_send_lat` to verify raw link performance between nodes

3

Infrastructure Metrics

Monitor switch/NIC counters for **drops**, **ECN marks**, **PFC pauses**, and queue depth

4

Application Telemetry

Measure All-Reduce durations, **p99 step time**, and GPU utilization patterns during training



Quick Wins Checklist



MTU Configuration

Set **jumbo frames (MTU 9000)** everywhere, or ensure consistent MTU sizes

NCCL Interface Binding

Pin NCCL to the correct interface/NIC; disable slow fallback paths

Ring Homogeneity

Keep communication rings **homogeneous** with consistent GPU/NIC speeds

Topology Awareness

Implement rack-aware scheduling; avoid spanning weak inter-rack links

Gradient Optimization

Increase DDP bucket size; enable AMP; use gradient accumulation

Advanced Moves

Hierarchical All-Reduce

Implement two-phase reduction (intra-node → inter-node) to optimize for physical topology

In-Network Reduction

Leverage SHARP-style hardware acceleration if available on your network fabric

Topology-Aware Launchers

Use MPI hostfiles, Ray placement groups, or SLURM topology hints for optimal rank mapping

Traffic Isolation

Configure dedicated QoS class for training traffic; isolate from storage and other bursty flows

Triage Playbook

Systematic Order of Operations

Validate Raw Performance

Measure baseline bandwidth/latency with iperf3 and RDMA tests between all node pairs

Verify Hardware Configuration

Check NUMA/NVLink topology and GPUDirect RDMA wiring with `nvidia-smi topo -m`

Optimize Framework Settings

Tune NCCL/Horovod bucket sizes and compute/communication overlap parameters

Address Network Configuration

Fix MTU, QoS, ECN/PFC settings and review oversubscription ratios in switch fabric

Enhance Data Access

Optimize data pipeline efficiency and storage locality to eliminate stalls

Measure & Iterate

Re-measure with **nccl-tests** followed by real training runs to verify improvements

Key Takeaways

Root Causes

Most "GPU slowness" stems from **collective communication bottlenecks** and suboptimal topology mapping

Three-Pronged Approach

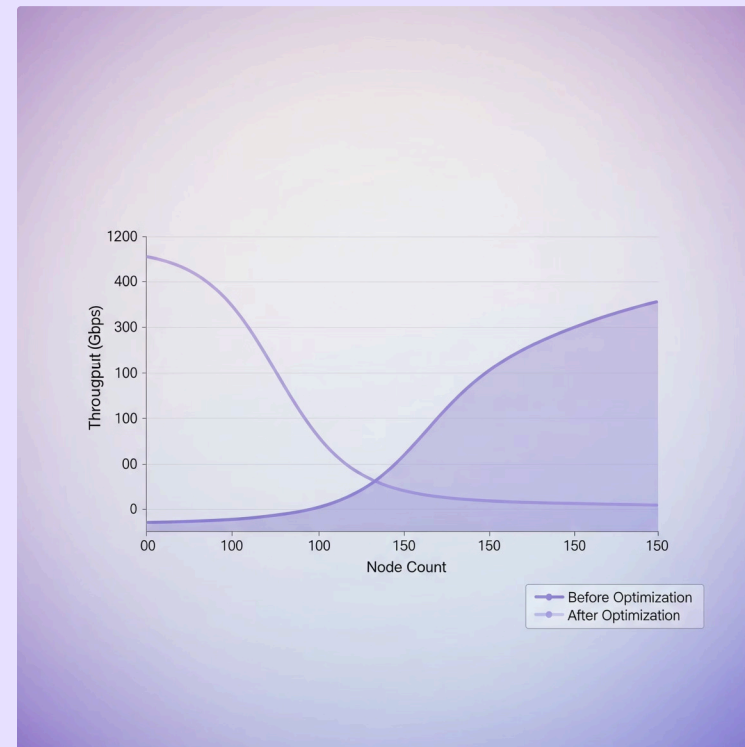
Fix the **bytes** (volume), the **paths** (topology), and the **placement** (locality)

Measure What Matters

Focus on **p99 step time** and All-Reduce durations, not just averages

Compounding Returns

Small, disciplined tweaks can yield large, repeatable scaling gains



Contact: training-infra@company.com

Internal docs: go/dist-training-network